SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: **Software Testing** **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*
Assessment Tool Weightage: 50%

Code of Conduct:

I ***HEMANTH KUMAR B (192421263)*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Hemanth Kumar B

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

SOFTWARE TESTING – DESCRIPTIVE WRITTEN EXAM

1. Testing as an Engineering Activity

Introduction

Software testing is not simply running a few test cases after the software has been developed. It is a systematic engineering activity that includes planning, requirement analysis, test design, execution, defect reporting, and continuous improvement.

This approach is very important in banking applications because a software failure can affect money transfers, account balances, customer information, and user trust.

Testing Approach

A proper testing process can be followed as:

Requirement Analysis → Risk Identification → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing

First, the testing team should understand all the requirements. Important banking functions such as login, account balance, fund transfer, withdrawal, and transaction history should be given higher priority.

Instead of testing randomly, testers can use proper test design techniques such as:

1. **Equivalence Partitioning** – Divides input data into valid and invalid groups.
2. **Boundary Value Analysis** – Tests values at and around the limits.
3. **Decision Table Testing** – Checks different combinations of conditions and actions.
4. **State Transition Testing** – Checks how the system behaves when it moves between different states.
5. **Risk-Based Testing** – Gives more attention to high-risk banking operations.
6. **Regression Testing** – Checks whether a change has affected existing functionality.

Example

Consider a fund-transfer feature. The tester should test more than just a successful transfer. They should also check:

- Valid transfer amount
- Insufficient balance
- Invalid beneficiary
- Maximum transaction limit
- Duplicate transaction
- Network interruption
- Transaction timeout
- Unauthorized transaction

For example, if an account has ₹25,000 and the customer transfers ₹10,000, the system should deduct exactly ₹10,000 and show ₹15,000 as the remaining balance. The transaction should also appear correctly in the transaction history.

Conclusion

Treating testing as an engineering activity makes testing more organized instead of random. It improves defect detection, requirement coverage, defect tracking, and regression testing. This ultimately helps in building a more reliable banking application.

2. Structured Testing Process Across the SDLC

An ATM system handles important activities such as card authentication, account verification, cash withdrawal, and financial transactions. Therefore, testing should be performed throughout the Software Development Life Cycle (SDLC), not only after coding is completed.

Requirement and Planning Stage

At the beginning, testers should carefully review requirements related to:

- Card validation
- PIN authentication
- Account verification
- Balance enquiry
- Cash withdrawal
- Transaction limits
- Receipt generation
- Error handling

This helps identify unclear or missing requirements before they become actual software defects.

Test Design and Implementation

After the requirements are finalized, test cases should be prepared. Both positive and negative scenarios should be included.

For example, ATM withdrawal testing can include:

Test Condition	Expected Result
Valid card and PIN	Transaction proceeds
Invalid PIN	Transaction is rejected
Insufficient balance	Withdrawal is rejected
Amount above limit	Transaction is rejected
Invalid amount	Error message is displayed
ATM has insufficient cash	Transaction is handled safely
Network failure	Incorrect deduction should not occur

Execution and Defect Management

During test execution, testers compare the actual result with the expected result. If there is a difference, it is reported as a defect.

After the developer fixes the defect, the tester performs **retesting**. Then **regression testing** is performed to make sure the fix has not affected other working features.

Conclusion

A structured testing process across the SDLC helps identify defects early. It improves transaction accuracy, security, reliability, and user confidence in an ATM system.

3. Errors, Defects and Failures

Errors, defects, and failures are related, but they do not mean the same thing. Understanding the difference is important for proper software testing and defect reporting.

Error

An **error** is a human mistake made during requirement analysis, design, coding, or another development activity.

Defect

A **defect** is the flaw introduced into the software because of the error.

Failure

A **failure** is the incorrect behavior that the user or tester observes when the defective software is executed.

The relationship can be represented as:

Human Error → Software Defect → Execution → Failure

Example

Suppose the requirement says that a password must contain at least eight characters, but a developer understands it incorrectly and implements a five-character minimum.

In this case:

- The misunderstanding of the requirement is the **error**.
- The incorrect password validation logic is the **defect**.
- Allowing a five-character password is the **failure**.

Another example is a transaction where the customer expects ₹5,000 to be transferred, but the system deducts ₹6,000. The tester observes the incorrect deduction and reports the failure.

Defect Reporting

A good defect report should contain:

1. Defect title
2. Environment
3. Preconditions
4. Test data
5. Steps to reproduce
6. Expected result
7. Actual result
8. Severity and priority
9. Screenshots or logs, if required

Conclusion

In simple terms, an **error is the human mistake, a defect is the software problem caused by that mistake, and a failure is the incorrect result seen during execution**. Understanding these terms makes defect reporting and communication much clearer.

4. Defect Clustering and Test Prioritization

Defect clustering is a software testing principle which means that a large number of defects are often concentrated in a small number of modules.

Therefore, testing teams should use historical defect information to identify areas that are more likely to contain problems.

Identifying Defect-Prone Modules

Modules can be considered high-risk when they have:

- High defect frequency
- High-severity defects
- Complex logic
- Frequent changes
- Many dependencies
- High customer usage

For example, in an online banking system, the fund-transfer module may contain more defects than a simple profile-display module because it involves more validations and integrations.

Test Prioritization

All test cases do not need to have the same priority.

A simple priority structure can be:

Priority 1 – Critical: Payment, fund transfer, authentication

Priority 2 – Important: Account statement, notifications

Priority 3 – Low Risk: Cosmetic or rarely used features

Suppose historical data shows that 60% of serious defects occurred in the payment module. The testing team should increase coverage for successful payments, failed payments, duplicate payments, timeouts, network interruptions, invalid card details, and refunds.

Conclusion

Defect clustering helps testers use their limited time and resources effectively. However, low-defect modules should not be completely ignored. Historical defect information should be combined with risk, complexity, and business importance to decide testing priorities.

5. Defect Classification and Repository Utilization

A defect repository is useful when defects are properly classified and analyzed. Simply storing old defects is not enough. The information should be used to prevent similar problems in future releases.

Major Defect Categories

Defects can be classified into:

- Requirement defects
- Design defects
- Coding defects
- Integration defects
- Interface defects
- Data defects
- Performance defects
- Security defects

This classification helps the team understand where defects are coming from.

Defect Repository

A defect record can contain:

Defect ID | Module | Description | Severity | Priority | Root Cause | Status | Resolution

For example, if several releases contain payment-calculation defects, the team can identify payment processing as a defect-prone area.

Preventing Repeated Defects

The team should:

1. Perform root-cause analysis.
2. Convert important defects into regression test cases.
3. Improve requirement reviews.
4. Increase testing in defect-prone modules.
5. Automate repetitive tests.
6. Conduct code and design reviews.

Conclusion

A defect repository should not be treated only as a storage location. It should become a knowledge base that helps testers learn from previous problems and prevent similar defects in future releases.

6. Mobile Banking Application – Fund Transfer Failures

Frequent crashes during fund transfer indicate that the application may not have been tested under enough combinations of devices, networks, transactions, and application conditions.

Since fund transfer is a critical banking function, the problem needs systematic investigation.

Initial Investigation

The tester should collect information about:

- Device model
- Operating system version
- Application version
- Network condition
- Account balance
- Transfer amount

- Beneficiary details
- Steps performed before the crash
- Error messages
- Application and server logs

The same scenario should also be reproduced several times to determine whether the crash happens consistently.

Testing Areas

Functional Testing: Checks normal and abnormal transfer operations.

Integration Testing: Checks communication between the mobile interface, authentication service, transaction service, and database.

Network Testing: Checks weak networks, disconnections, reconnections, and timeouts.

Performance Testing: Checks the application during multiple simultaneous transactions.

Regression Testing: Ensures that fixes do not break previously working functions.

Example

Suppose the application crashes when the user presses the **Transfer** button while the network changes from strong to weak.

Possible causes could include:

- Poor exception handling
- Timeout handling problems
- Incorrect transaction state
- API communication problems

Improvements

The organization should introduce:

- Automated regression testing
- Network-condition testing
- Crash monitoring
- Device compatibility testing
- Exception-handling tests
- Transaction recovery testing

Conclusion

A systematic testing approach can help identify the actual reason for mobile banking crashes and improve the reliability of fund-transfer operations.

7. Testing and Debugging

Testing and debugging are closely related, but they are different activities.

Testing

Testing checks whether the software behaves according to the expected requirements. It helps identify failures.

Debugging

Debugging starts after a failure is identified. It focuses on finding the actual cause of the problem and correcting the defect.

Testing	**Debugging**
Detects failures	Finds the cause
Mainly performed by testers	Mainly performed by developers
Uses test cases	Uses code analysis, logs and debugging tools
Produces defect reports	Produces corrections
Checks expected behavior	Investigates internal behavior

Example

Suppose an ATM deducts ₹5,000 from an account but dispenses only ₹4,000.

The tester identifies the mismatch and reports it. The developer then investigates the cash-dispensing logic, transaction calculation, or communication with the ATM hardware.

After the developer fixes the problem:

Retesting → Regression Testing

Recommended Workflow

Test → Detect Failure → Report Defect → Debug → Fix → Retest → Regression Test

Conclusion

Testing shows that a problem exists, while debugging finds and removes its cause. Both activities need to work together for reliable software.

8. Cross-Browser Testing for an E-Commerce Platform

An e-commerce website may be used on different browsers, browser versions, operating systems, and devices. Because browsers can handle JavaScript, CSS, and screen layouts differently, the same website may not behave exactly the same everywhere.

Testing Strategy

The testing team should first prepare a **browser compatibility matrix** containing the browsers and versions officially supported by the application.

Important features to test include:

- Login

- Product search
- Product display
- Shopping cart
- Checkout
- Payment
- Order confirmation
- User profile

Common Causes of Cross-Browser Defects

1. JavaScript incompatibility
2. CSS rendering differences
3. Browser-specific features
4. Responsive design problems
5. Third-party component behavior
6. Different browser versions

Example

Suppose the checkout button works correctly in one browser but does not respond in another.

The tester should:

- Reproduce the issue
- Record the browser and version
- Check console errors
- Inspect network requests
- Compare behavior across supported browsers
- Capture screenshots and other evidence

Improving Coverage

The team can use:

- Automated browser testing

- Compatibility matrices
- Responsive testing
- Manual exploratory testing
- Regression testing
- Browser-based test environments

Conclusion

Cross-browser testing helps ensure that users get consistent functionality across supported environments. Combining automation with manual and regression testing provides better coverage.

9. Defect Classification in Payroll Systems

If a payroll system calculates an employee's salary incorrectly because of faulty program logic, the problem is mainly a **coding or logical defect**. However, the tester should also check whether the coding problem was caused by an incorrect requirement or design.

Possible Sources

Requirement Level: Salary rules may have been specified incorrectly.

Design Level: The salary calculation model may be wrong.

Coding Level: The implemented formula may contain a logical error.

Data Level: Incorrect salary or deduction information may be processed.

Suitable Testing Techniques

Equivalence Partitioning: Employees can be divided into different categories based on salary or employment conditions.

Boundary Value Analysis: Salary, tax, and deduction limits should be tested around their boundaries.

Decision Tables: Useful when salary calculation depends on multiple conditions.

Regression Testing: Ensures that correcting one calculation does not affect other salary calculations.

Example

Suppose:

- Basic Salary = ₹30,000
- Allowances = ₹5,000
- Deductions = ₹3,000

Expected net salary:

₹30,000 + ₹5,000 − ₹3,000 = ₹32,000

If the application calculates ₹28,000, the tester should investigate the calculation logic and identify the underlying defect.

Conclusion

Proper defect classification helps testers choose the right testing techniques. Requirement reviews, code reviews, boundary testing, automated calculation tests, and regression testing can improve payroll accuracy.

10. Structured Test Case Design

Random testing may find obvious problems, but it does not provide confidence that all important requirements have been tested. Therefore, test cases should be designed systematically before execution.

Components of a Good Test Case

A proper test case should contain:

1. Test Case ID
2. Objective

3. Preconditions
4. Input data
5. Test steps
6. Expected result
7. Actual result
8. Pass/Fail status
9. Defect reference, if required

Test Design Techniques

The following techniques can improve test effectiveness:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Error Guessing
- Risk-Based Testing

Example – Login Function

Instead of entering random usernames and passwords, planned test cases can include:

Condition	**Expected Behavior**
Valid username + valid password	Login successful
Invalid username	Login rejected
Invalid password	Login rejected
Empty fields	Validation message displayed
Locked account	Access denied
Repeated invalid attempts	Appropriate security response

Advantages

Structured test cases provide:

- Better requirement coverage
- Repeatable execution
- Clear expected results
- Easier defect identification
- Better regression testing
- Improved traceability

Conclusion

Structured test case design changes testing from random checking into a planned activity. It helps testers find more defects and provides clear evidence about software quality.

11. Data Integrity in Healthcare Applications

Healthcare applications usually contain several connected modules, such as patient registration, appointments, laboratory services, pharmacy, billing, and medical records.

If patient information changes incorrectly between these modules, the reliability of the entire system can be affected.

Possible Sources of Data Problems

Data defects may be caused by:

- Incorrect requirements
- Database design problems
- Incorrect field mapping
- Duplicate records
- Integration errors
- Incorrect update logic
- Data transformation problems

Systematic Validation

Patient information should be followed through the complete workflow:

Registration → Appointment → Laboratory → Pharmacy → Billing

Important information such as patient ID, name, date of birth, test information, and billing details should remain correct wherever they are used.

Example

Suppose a patient's date of birth is correctly entered during registration but appears differently in the billing module.

The tester should investigate whether the problem occurred during:

1. Data storage
2. Data transfer
3. Database retrieval
4. Data conversion
5. User-interface display

Improvements

The team should use:

- Integration testing
- Database validation
- Data consistency checks
- Regression testing
- Detailed defect reports
- Requirement traceability

Conclusion

Data-integrity testing ensures that patient information remains accurate while moving between different healthcare modules. Proper integration testing and defect tracking are important for maintaining reliable healthcare software.

12. Defect Repository and Recurring Defects

Repeated defects often indicate that previous testing information is not being used effectively. A defect repository can help the team study old problems and convert them into useful preventive test cases.

Information to Record

Each defect record should contain:

Defect ID | Module | Description | Severity | Priority | Root Cause | Status | Resolution

This information allows the organization to analyze defects across different releases.

Identifying Patterns

The team can study defects based on:

- Module
- Severity
- Release
- Root cause
- Frequently recurring defect types

For example, if discount-related defects repeatedly occur in an e-commerce system, the team should create additional test cases for different discount combinations.

Preventive Strategy

The organization should:

1. Perform root-cause analysis.
2. Improve requirement reviews.
3. Increase testing in defect-prone modules.
4. Review previous defects before new releases.
5. Automate important regression scenarios.
6. Add important historical defects to the regression suite.

Conclusion

A defect repository should be used as a source of testing knowledge rather than simply a place to store old defects. Studying historical patterns helps prevent recurring problems and improves future software releases.

13. Peak-Usage Failures in a Web-Based Learning System

A web-based learning system may work properly when only a small number of users are online but fail when thousands of students access it at the same time.

This can happen during examinations, result publication, or assignment submission. It usually indicates insufficient performance and workload testing.

Possible Testing Gaps

The problems may be caused by:

- Insufficient load testing
- Lack of stress testing
- Unrealistic test data
- Limited concurrent-user testing
- Inadequate database testing

- Poor performance monitoring
- Missing recovery scenarios

Suitable Testing

Load Testing: Checks system behavior under the expected number of users.

Stress Testing: Checks what happens when the system goes beyond its normal capacity.

Performance Testing: Measures response time, throughput, and resource usage.

Recovery Testing: Checks whether the system can recover after overload or failure.

Example

Suppose an online examination system works properly with 100 users but fails when 5,000 students submit answers at the same time.

The testers should monitor:

- Response time
- CPU utilization
- Memory consumption
- Database performance
- Network activity
- Error rate
- Transaction completion

Improvements

The team should create realistic peak-load scenarios and repeat them before major releases. Automated performance testing can also simulate large numbers of users.

Conclusion

Performance failures during peak usage usually happen when real-world workload conditions are not properly tested. Load, stress, performance, and recovery testing should therefore be included in the testing strategy.

14. Early Involvement of Testers in the SDLC

Testers should not be brought into a project only after development is finished. Their involvement from the requirement stage helps identify defects and unclear requirements much earlier.

Role of Testers at Different Stages

Requirement Stage

Testers can:

- Review requirements
- Identify unclear statements
- Check whether requirements are testable
- Find missing conditions

Design Stage

They can:

- Review workflows
- Identify integration risks
- Analyze data flow
- Identify possible failure conditions

Development Stage

They can:

- Prepare test data
- Review testability
- Support unit-test planning
- Identify automation opportunities

Testing Stage

They execute tests, report defects, perform retesting, and conduct regression testing.

Example

Consider a banking fund-transfer requirement. A tester can ask questions such as:

- What happens if the balance is insufficient?
- What happens if the beneficiary account is invalid?
- What happens if the network fails?
- What happens if the user submits the transaction twice?
- What happens if the transaction limit is exceeded?

These questions help clarify requirements before development begins.

Benefits

Early tester involvement provides:

- Earlier defect detection
- Lower rework cost
- Better requirements
- Improved testability
- Better risk identification
- Stronger collaboration

Conclusion

Early tester participation supports the **shift-left testing approach**, where testing and quality activities begin earlier in the SDLC. This helps prevent defects instead of only finding them later.

15. Improving Incomplete Test Coverage

A software system may work correctly for common situations but fail under unusual conditions. This does not necessarily mean the software is reliable; it may indicate that the testing coverage is incomplete.

Identifying Coverage Gaps

The testing team should compare the requirements with the existing test cases.

Coverage gaps can occur in:

- Functional requirements
- Negative scenarios
- Boundary conditions
- Error handling
- Integration paths
- Previously fixed defects
- Exceptional workflows

Improving Test Design

The following techniques can improve coverage:

Equivalence Partitioning – Identifies representative valid and invalid inputs.

Boundary Value Analysis – Focuses on minimum and maximum values.

Decision Tables – Tests different combinations of conditions.

State Transition Testing – Checks behavior when the system moves between states.

Example

Suppose an application accepts values from **1 to 100**.

Important test values include:

0, 1, 2, 99, 100, 101

Testing only the value 50 is not enough because it does not properly test the boundaries.

Recommended Actions

The team should:

1. Create a requirement traceability matrix.
2. Identify missing test cases.
3. Add negative and boundary scenarios.
4. Add historical defects to regression testing.
5. Automate repetitive high-priority cases.
6. Review test coverage before release.

Conclusion

Complete test coverage cannot guarantee that software has no defects. However, it increases confidence that important requirements and conditions have been properly tested.

16. Improving Defect Reporting

Poor defect reporting can make it difficult for developers to reproduce and fix a problem. Therefore, defect reports should contain enough information to clearly explain what happened.

This is especially important in a logistics application involving orders, inventory, warehouses, and deliveries.

Information in a Good Defect Report

A professional defect report should contain:

- **Defect ID** – Unique identification number
- **Title** – Short description of the problem
- **Environment** – Browser, operating system, application version, etc.

- **Preconditions** – Conditions required before testing
- **Steps to Reproduce** – Exact steps to reproduce the problem
- **Expected Result** – What should have happened
- **Actual Result** – What actually happened
- **Severity and Priority** – Impact and urgency
- **Evidence** – Screenshots, logs, or videos when necessary

Example

Instead of simply writing:

“Delivery date is wrong.”

A better defect report should mention the order number, delivery location, selected date, expected date, actual date, steps performed, and application version.

This gives the developer enough information to reproduce the issue quickly.

Defect Lifecycle

A typical defect lifecycle is:

New → Assigned → In Progress → Fixed → Retest → Closed

If the defect still exists after retesting, it can be reopened.

Benefits

Structured defect reporting improves:

- Reproducibility
- Communication
- Tracking
- Resolution speed
- Root-cause analysis
- Historical analysis

Conclusion

A clear defect report converts a tester's observation into useful technical information. Standardized reporting helps developers understand and resolve problems more quickly.

17. Process-Based Testing Across Modules

When different teams test different modules using different procedures, the results may become inconsistent. Therefore, a common testing process should be followed across the project.

Common Testing Workflow

A standard process can be:

Requirement Analysis
↓
Test Planning
↓
Test Case Design
↓
Environment Preparation
↓
Test Execution
↓
Defect Reporting
↓
Retesting
↓
Regression Testing

Every module should follow this basic process.

Standard Test Documentation

A common test-case format should contain:

- Test Case ID
- Objective
- Preconditions
- Test Data
- Steps
- Expected Result
- Actual Result
- Status

Defect reports should also follow a standard format.

Example

Consider an e-commerce system with:

Login → Product Search → Shopping Cart → Payment

All these modules can follow the same basic testing process.

However, the payment module should receive additional security, transaction, and integration testing because it carries greater risk.

Maintaining Consistency

Consistency can be improved through:

1. Standard operating procedures
2. Common test templates
3. Shared test-management tools
4. Common severity definitions
5. Test-case reviews
6. Shared regression suites

7. Automation guidelines

Conclusion

Process-based testing provides a consistent foundation for testing different modules. At the same time, extra testing can be added to high-risk modules. This improves traceability, repeatability, and overall software quality.

18. Boundary Testing and Untested Edge Cases

Many software applications work correctly with normal inputs but fail when values are close to their limits. These unusual situations are called **edge cases**.

Why Edge Cases Are Missed

They may be missed because of:

- Incomplete requirements
- Limited test data
- Time constraints
- Focus mainly on positive scenarios
- Lack of boundary analysis
- Insufficient historical defect analysis

Boundary Value Analysis

Boundary Value Analysis focuses on values at and around the limits.

For example, suppose an ATM has a maximum daily withdrawal limit of **₹50,000**.

Useful test cases are:

- **₹49,999** – Just below the limit
- **₹50,000** – Exact limit

- **₹50,001** – Just above the limit

These tests are more useful for boundary checking than testing only an ordinary value such as ₹10,000.

Other Edge Cases

Testing should also include:

- Empty inputs
- Zero values
- Maximum field length
- Minimum values
- Maximum values
- Invalid dates
- Expired information
- Duplicate requests
- Network interruption

Improving Test Design

Testers should identify boundaries directly from the requirements and convert them into test cases. Important edge cases should also be added to regression testing after a defect has been fixed.

Conclusion

Boundary testing is an effective technique for finding defects that may remain hidden during normal testing. Including edge cases systematically improves reliability and reduces production failures.

19. Defect Documentation and Repository Management

When defects are not properly documented, important information about what went wrong, how it happened, and how it was fixed can be lost.

A structured defect repository provides a central record of defects throughout their lifecycle.

Information to Maintain

Information	Purpose
Defect ID	Unique identification
Title	Quick understanding
Module	Identifies affected area
Description	Explains the problem
Severity	Measures impact
Priority	Determines urgency
Steps	Helps reproduce the problem
Expected Result	Defines correct behavior
Actual Result	Records observed behavior
Root Cause	Supports prevention
Resolution	Records the fix
Status	Tracks lifecycle

Defect Lifecycle

A typical lifecycle is:

New → Assigned → In Progress → Fixed → Retest → Closed

If the problem still exists, the defect can be reopened.

Repository Analysis

The repository can be analyzed based on:

- Module
- Severity
- Priority
- Release
- Root cause
- Recurring defect type

For example, repeated payment defects may indicate weaknesses in transaction validation or integration.

Benefits

Proper documentation provides:

- Better traceability
- Faster reproduction
- Improved communication
- Historical knowledge
- Better root-cause analysis
- Stronger regression testing

Conclusion

A defect repository should work as both a tracking system and a source of organizational knowledge. Studying old defects helps the team understand recurring problems and improve future testing.

20. Redesigning Testing for Banking Transaction Processing

Repeated failures in banking transaction processing indicate that the existing testing approach needs improvement. Since banking transactions involve financial information, testing should cover functional correctness, integration, security, performance, boundaries, and recovery.

Proposed Testing Model

A suitable testing process is:

Requirement Analysis → Risk Assessment → Test Design → Functional Testing → Integration Testing → Performance Testing → Security Testing → Regression Testing

Critical transaction requirements should be identified clearly before designing the test cases.

Important Test Conditions

The testing process should cover:

- Valid transaction
- Insufficient balance
- Invalid account
- Invalid authentication
- Transaction-limit violation
- Duplicate transaction
- Network failure
- Server timeout
- Transaction cancellation
- Recovery after failure

Example

Suppose **Account A contains ₹20,000** and the customer transfers **₹5,000 to Account B**.

The expected result is:

1. ₹5,000 should be deducted from Account A.
2. ₹5,000 should be credited to Account B.
3. Both account balances should be updated correctly.
4. Only one transaction record should be created.
5. The correct transaction status should be displayed.
6. A confirmation should be provided to the customer.

If the network fails during processing, the system should not create a duplicate or partial transaction.

Test Design Techniques

Equivalence Partitioning can separate valid and invalid transaction inputs.

Boundary Value Analysis can be used to test transaction limits.

Decision Tables can test combinations involving account status, balance, transaction amount, and authorization.

State Transition Testing can check transaction states such as:

Initiated → Processing → Successful / Failed / Cancelled

Improving Defect Detection

The bank should introduce:

- Automated regression testing
- Risk-based test prioritization
- Integration testing
- Performance testing
- Security testing
- Network-failure testing
- Transaction recovery testing
- Historical defect analysis

Retesting and Regression Testing

Whenever a defect is fixed, the original scenario should first be retested. After that, regression testing should be performed on related transaction functions to make sure the correction has not created another problem.

Conclusion

Banking transaction systems require a structured and risk-based testing approach. By combining systematic test design, integration, boundary, performance, security, recovery, defect tracking, and regression testing, the bank can improve transaction reliability and reduce repeated failures.